

ND-agent scheduling of linear-deteriorating tasks with positional due indices to minimize total completion time and maximum cost



Rubing Chen, Jinjiang Yuan*, Zhichao Geng

School of Mathematics and Statistics, Zhengzhou University, Zhengzhou 450001, People's Republic of China

ARTICLE INFO

Article history:

Received 28 March 2019

Revised 23 June 2019

Accepted 26 August 2019

Keywords:

ND-agent scheduling

Linear-deteriorating tasks

Due indices

Pareto scheduling

ABSTRACT

This paper investigates the ND-agent scheduling of linear-deteriorating tasks on a single machine with positional due indices. Under the ND-agent setting, there are two agents A and B and each agent $X \in \{A, B\}$ has its set of tasks $\mathcal{T}^{(X)}$, called X -tasks. Moreover, ND-agent means that $\mathcal{T}^{(A)}$ and $\mathcal{T}^{(B)}$ are allowed to be non-disjoint, which is a generalization of the traditional two-agent setting. Each task has a linear-deteriorating processing time and a positional due index. In this paper, we consider two problems: The first problem is to find a feasible schedule which minimizes the total completion time of the A -tasks under the condition that the maximum cost of the B -tasks is bounded by a threshold value. The second problem is the Pareto scheduling problem to minimize the total completion time of the A -tasks and the maximum cost of the B -tasks, simultaneously. We show in this paper that the two problems are solvable in $O(n^2)$ time and in $O(n^4)$ time, respectively. If the maximum cost of the tasks of agent B is restricted to some lateness-like criteria, for the version without the positional restriction, the time complexity for solving the above two problems can be reduced to $O(n \log n)$ and $O(n^3 \log n)$, respectively. Our research includes a new technique for calculating the number of non-dominated points.

© 2019 Elsevier Inc. All rights reserved.

1. Introduction

We study the ND-agent scheduling problems on a single machine, where each task has a linear-deteriorating processing time and a positional due index. Our research combines three active scheduling models together for minimizing the total completion time and the maximum scheduling cost.

Although the scheduling with multiple agents, deteriorating tasks, or positional due indices has been widely investigated in the literature, known research has seldom considered the three scheduling topics simultaneously. Such a kind of scheduling model may appear in the steel industry. The molten steel from steel furnace is processed into steel products needed for production and living. Before rolling the ingots, the ingots are needed to reheat to a pretty high temperature. The temperature of the ingots will decrease when waiting time increases. Due to the temperature decreases, the processing times of ingots will increase. The steel furnace can be regarded as a single machine. Different customers can be regarded as multiple agents. Moreover, each order (task) may have its own expected position number in the processing procedure. Then scheduling with multiple agents, deteriorating tasks and positional due indices phenomena appears in this practical application.

* Corresponding author.

E-mail address: yuanjj@zzu.edu.cn (J. Yuan).

Problem formulation: Suppose that we are given a set of n linear-deteriorating tasks $\mathcal{T} = \{1, 2, \dots, n\}$ to be scheduled on a single machine. In our scheduling model, “linear-deteriorating” means that the *actual processing time* of each task j which starts at time t in a schedule is of the form $p_j(t) = \alpha_j(a + bt)$, where $a \geq 0$, $b \geq 0$, $a + b > 0$, and $\alpha_j > 0$ is the *normal processing time* of task j which indicates the part in $p_j(t)$ only related to task j . In the literature, α_j is also called the *deterioration rate* of task j . It is reasonable to assume that all tasks are ready for processing at a common time $\tau_0 \geq 0$. When $a = 0$, we also assume that $\tau_0 > 0$. The research in Kononov and Gawiejnowicz [18] tells us that, if $\mathcal{X}$ is an arbitrary subset of $\mathcal{T}$ and the tasks in $\mathcal{X}$ are consecutively processed (in an arbitrary order) starting at time t , then the last task in $\mathcal{X}$ is completed at time

$$C(\mathcal{X}, t) = \left(t + \frac{a}{b}\right) \prod_{j \in \mathcal{X}} (1 + b\alpha_j) - \frac{a}{b}. \quad (1)$$

Note that we may regard b as a formal parameter. Then (1) is still valid for $b = 0$. This equation is very useful since it enables us to focus on the schedules without idle time, starting at time τ_0 and completing at time $C(\mathcal{T}, \tau_0)$ in the sequel. In this paper, we also assume that each task j has a positional due index k_j , where “positional due index” means that the position number of each task cannot exceed its positional due index in any feasible schedule. Under the two-agent setting, there are two agents A and B and each agent $X \in \{A, B\}$ has its set of tasks $\mathcal{T}^{(X)}$, called X -tasks. Generally, we require that $\mathcal{T}^{(A)} \cup \mathcal{T}^{(B)} = \mathcal{T}$. In the case that $\mathcal{T}^{(A)}$ and $\mathcal{T}^{(B)}$ are disjoint, the two agents are called *competitive agents* and this setting is called *CO-agent*, so we put in the β -field “CO” to denote the CO-agent setting. Alternatively, if $\mathcal{T}^{(A)}$ and $\mathcal{T}^{(B)}$ are not necessarily disjoint, the two agents are called *non-disjoint agents* and this setting is called *ND-agent*, so we put in the β -field “ND” to denote the ND-agent setting. Clearly, CO-agent setting is a special version of ND-agent setting.

Given a feasible schedule σ , the completion time of task j is denoted by $C_j(\sigma)$. Let $f_j(\cdot)$ be the cost function of task $j \in \mathcal{T}$, which depends on $C_j(\sigma)$. We only consider the cost functions $f_j(\cdot)$ which is nondecreasing in the time space $[0, +\infty)$. Let $\sum C_j^{(A)}(\sigma) = \sum_{j \in \mathcal{T}^{(A)}} C_j(\sigma)$ be the total completion time of A -tasks and $f_{\max}^{(B)}(\sigma) = \max_{j \in \mathcal{T}^{(B)}} f_j(C_j(\sigma))$ be the maximum cost of B -tasks in σ .

For each agent $X \in \{A, B\}$, let $f^{(X)}$ be the scheduling objective function of agent X to be minimized. The following two forms of ND-agent scheduling problems are what we studied in this paper:

- (i) 1|ND, k_j , $p_j(t)$ | $f^{(A)}$: $f^{(B)} \leq Q$. This is the constrained ND-agent scheduling problem which aims to minimize $f^{(A)}$ under the constraint $f^{(B)} \leq Q$.
- (ii) 1|ND, k_j , $p_j(t)$ | $f^{(A)}, f^{(B)}$. This is the Pareto ND-agent scheduling problem which aims to minimize $f^{(A)}$ and $f^{(B)}$, simultaneously. Readers can refer to T'Kindt and Billaut [27] for more detail discussions on Pareto scheduling problems.

In this paper, we mainly consider the following two ND-agent scheduling problems:

$$1|\text{ND}, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q, \quad (2)$$

and

$$1|\text{ND}, k_j, p_j(t)|^{\#} \left(\sum C_j^{(A)}, f_{\max}^{(B)} \right). \quad (3)$$

Our research is also valid for the two CO-agent scheduling problems

$$1|\text{CO}, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$$

and

$$1|\text{CO}, k_j, p_j(t)|^{\#} \left(\sum C_j^{(A)}, f_{\max}^{(B)} \right)$$

since they are the special versions of the two problems in (2) and (3), respectively.

Known results: In the research for rescheduling problems, Zhao and Yuan [43] introduced the concept of positional due indices of tasks, which requires that each task j must be scheduled in a position no later than a previously given index k_j . Gao and Yuan [11,12] presented some basic research for the single-machine scheduling with positional due indices. Up to now, this topic has not been extensively studied. The results in Gao and Yuan [11,12] related to the research of this paper are summarized in Table 1.

Scheduling with deteriorating tasks was first studied by Melnikov and Shafransky [22]. Browne and Yechiali [7] introduced the scheduling problems in which the actual processing time of any task is a function of its starting time. Alidaee and Womer [4] and Cheng et al. [8] presented extensive study in the analysis of scheduling problems related to deteriorating tasks. Research on scheduling with deteriorating tasks in the flow shop model can be found in [24,32]. Research on scheduling with deteriorating tasks under graph or chain precedence constraints can be found in [28–31]. Recent survey on scheduling with deteriorating tasks can be found in Agnetis et al. [1] and Strusevich and Rustogi [25]. The results in this research topic related to the research of this paper are summarized in Table 1.

Baker and Smith [6] and Agnetis et al. [2] were the pioneers that brought the two-agent problems into scheduling field. Their two-agent setting assumes that the two agents have disjoint sets of tasks, and so, is in fact the CO-agent version. Research on multi-agent scheduling is a very hot topic in the past decades. Earlier work on two-agent (or multi-agent)

Table 1

The known results related to the research in this paper.

Scheduling problems	Time complexity	References
$1 k_j \Sigma C_j$	$O(n \log n)$	Zhao et al. [42]
$1 k_j L_{\max}$	$O(n \log n)$	Zhao and Yuan [43]
$1 k_j, \text{prec} f_{\max}$	$O(n^2)$	Zhao and Yuan [43]
$1 k_j ^{\#}(\Sigma C_j, f_{\max})$	$O(n^4)$	Gao and Yuan [11]
$1 k_j, \text{prec} ^{\#}(f_{\max}, g_{\max})$	$O(n^4)$	Gao and Yuan [12]
$1 p_j(t) = \alpha_j(a + bt) L_{\max}$	$O(n \log n)$	Bachman and Janiak [5]
$1 p_j(t) = \alpha_j(a + bt) C_{\max}$	$O(n)$	Zhao et al. [41]
$1 p_j(t) = \alpha_j(a + bt) f_{\max}$	$O(n^2)$	Zhao et al. [41]
$1 p_j(t) = \alpha_j(a + bt) \Sigma w_j C_j$	$O(n \log n)$	Zhao et al. [41]
$1 \text{ND} \Sigma C_j^{(A)} : f_{\max}^{(B)} \leq Q$	$O(n \log n)$	Agnetis et al. [1]
$1 \text{ND} ^{\#}(\Sigma C_j^{(A)}, f_{\max}^{(B)})$	$O(n^3 \log n)$	Agnetis et al. [1]
$1 \text{CO} \Sigma C_j^{(A)} : f_{\max}^{(B)} \leq Q$	$O(n \log n)$	Agnetis et al. [2]
$1 \text{CO}, k_j, \text{prec}^{(B)} ^{\#}(\Sigma C_j^{(A)}, f_{\max}^{(B)})$	$O(n^4)$	Gao and Yuan [12]
$1 \text{CO}, k_j ^{\#}(\Sigma C_j^{(A)}, L_{\max}^{(B)})$	$O(n^3 \log n)$	Gao and Yuan [12]
$1 \text{CO}, p_j(t) = \alpha_j(a + bt) \Sigma C_j^{(A)} : f_{\max}^{(B)} \leq Q$	$O(n \log n)$	Liu et al. [21]
$1 \text{CO}, p_j(t) = \alpha_j(a + bt) ^{\#}(\Sigma C_j^{(A)}, f_{\max}^{(B)})$	$O(n^4)$	He and Leung [14]
$1 \text{CO}, p_j(t) = \alpha_j(a + bt) \Sigma w_j^{(A)} C_j^{(A)} : f_{\max}^{(B)} \leq Q$ (under condition that $\alpha_i^{(A)} \leq \alpha_j^{(A)} \Rightarrow w_i^{(A)} \geq w_j^{(A)}$)	$O(n \log n)$	Yin et al. [34]
$1 \text{CO}, p_j(t) = \alpha_j(a + bt) f_{\max}^{(A)} : f_{\max}^{(B)} \leq Q$	$O(n^2)$	Yin et al. [34]
$1 \text{CO}, p_j = \alpha_j t L_{\max}^{(A)} : C_{\max}^{(B)} \leq Q$	$O(n \log n)$	Liu and Tang [20]
$1 \text{CO}, p_j = \alpha_j t \Sigma C_j^{(A)} : f_{\max}^{(B)} \leq Q$	$O(n \log n)$	Wang et al. [33]

scheduling can be found in [3,9,10,19,23,36]. After these achievements, some new multi-agent scheduling models were introduced. For example, Huynh-Tuong and Soukhal [16] and Huynh-Tuong et al. [17] introduced and studied the two-agent scheduling with a global objective function. The ND-agent scheduling was first introduced in the book by Agnetis et al. [1]. Note that the ND-agent scheduling is in fact a generation of the CO-agent scheduling. For various choices of the criteria $f^{(A)}$ and $f^{(B)}$, Agnetis et al. [1] presented complexity results for the problems $1|\text{ND}|f^{(A)} : f^{(B)} \leq Q$ and $1|\text{ND}|\alpha f^{(A)} + \beta f^{(B)}$. New complexity results on two-agent (or multi-agent) scheduling can be found in Yuan [35,37–39]. More results are surveyed in Perez-Gonzalez and Framinan [26] and Agnetis et al. [1]. The known results on two-agent scheduling most related to our research are displayed in Table 1.

It should be noticed that, in Table 1, the $O(n \log n)$ -time complexity for problems $1| - | - : f_{\max} \leq Q$ and the $O(n^3 \log n)$ -time complexity for problems $1| - |^{\#}(- : f_{\max})$ are based on the assumption that, for each task j , the induced deadline $\bar{d}_j := \max\{t : f_j(t) \leq Q\}$ can be determined in a constant time. Without this assumption, the time complexity should be (n^2) and $O(n^4)$, respectively. In this paper, we do not adopt such an assumption since it is invalid in general.

Our contributions: We show in this paper that the two ND-agent scheduling problems in (2) and (3) are solvable in $O(n^2)$ time and in $O(n^4)$ time, respectively. If $f_{\max}^{(B)}$ is restricted to some lateness-like criteria, i.e., $f_{\max}^{(B)} \in \{C_{\max}^{(B)}, WC_{\max}^{(B)}, L_{\max}^{(B)}, T_{\max}^{(B)}\}$, for the version without the positional restriction, the time complexity can be reduced to $O(n \log n)$ and $O(n^3 \log n)$, respectively.

Organization of this paper: This paper is organized as follows. In Section 2, we present some notations and basic lemmas. In Section 3, we study problem (2) and its subcases. In Section 4, we study problem (3) and its subcases. Conclusions are presented in the last section.

2. Some preliminaries

In Section 2.1, we introduce some notations. In Section 2.2, we give a formal definition of the Pareto optimization scheduling problem and present some useful principles.

2.1. Notations

Let $\{1, 2, \dots, n\}$ be the n tasks to be scheduled on a single machine. A schedule of the n tasks is given by a sequence $\sigma = (\sigma(1), \sigma(2), \dots, \sigma(n))$ of the n tasks, where $\sigma(i)$ is the task scheduled at the i th position in σ , $i = 1, 2, \dots, n$. We also call $\sigma(i)$ the i th task of the schedule σ . For each task $j \in \{1, 2, \dots, n\}$, we use k_j to denote the positional due index of task j and use $\sigma[j]$ to denote the position number of task j in a schedule σ . In a feasible schedule σ , we require that $\sigma[j] \leq k_j$ for all the tasks $j = 1, 2, \dots, n$.

Given a schedule $\sigma = (\sigma(1), \sigma(2), \dots, \sigma(n))$, we use $S_j(\sigma)$ and $C_j(\sigma)$ to denote the starting time and completion time of task j in σ , respectively, $j = 1, 2, \dots, n$. Then, for each $j = 1, 2, \dots, n$, the actual processing time of task j in schedule σ is given by

$$p_j(S_j(\sigma)) = \alpha_j(a + b \cdot S_j(\sigma)) \quad (4)$$

and the completion time of task j in σ is given by

$$C_j(\sigma) = S_j(\sigma) + p_j(\sigma) = S_j(\sigma) + \alpha_j(a + b \cdot S_j(\sigma)). \quad (5)$$

Since we only consider schedules which start at time τ_0 and have no artificial idles, from Eq. (1), the completion time of the i -th task $\sigma(i)$ in σ , denoted $C_{\sigma(i)}(\sigma)$, is given by

$$C_{\sigma(i)}(\sigma) = \left(\tau_0 + \frac{a}{b} \right) \prod_{x=1}^i (1 + b\alpha_{\sigma(x)}) - \frac{a}{b}, \quad i = 1, 2, \dots, n. \quad (6)$$

Moreover, the following notations will also be used in our discussion:

- $\sigma|_j$ is the subschedule of σ for the first j tasks.
- $L_j(\sigma) = C_j(\sigma) - d_j$ is the lateness of task j in σ .
- $T_j(\sigma) = \max\{0, C_j(\sigma) - d_j\} = \max\{0, L_j(\sigma)\}$ is the tardiness of task j in σ .
- $w_j C_j(\sigma)$ is the weighted completion time of task j in σ .
- $f_{\max}^{(B)}(\sigma) = \max_{j \in \mathcal{T}^{(B)}} f_j(C_j(\sigma))$ is the maximum cost of agent B in σ . We assume in this paper that, for each $j = 1, 2, \dots, n$, $f_j(t)$ is a nondecreasing function in the interval $[\tau_0, +\infty)$.
- $C_{\max}^{(B)}(\sigma) = \max_{j \in \mathcal{T}^{(B)}} C_j(\sigma)$ is the makespan of agent B in σ .
- $L_{\max}^{(B)}(\sigma) = \max_{j \in \mathcal{T}^{(B)}} L_j(\sigma)$ is the maximum lateness of agent B in σ .
- $T_{\max}^{(B)}(\sigma) = \max_{j \in \mathcal{T}^{(B)}} T_j(\sigma)$ is the maximum tardiness of agent B in σ .
- $WC_{\max}^{(B)}(\sigma) = \max_{j \in \mathcal{T}^{(B)}} w_j C_j(\sigma)$ is the maximum weighted completion time of agent B in σ .
- $\tilde{L}_{\max}^{(B)} \in \{C_{\max}^{(B)}, WC_{\max}^{(B)}, L_{\max}^{(B)}, T_{\max}^{(B)}\}$.
- $\sum C_j^{(A)}(\sigma) = \sum_{j \in \mathcal{T}^{(A)}} C_j(\sigma)$ is the total completion time of agent A in σ .

As introduced in Yuan et al. [40], the maximum cost criterion $f_{\max}$ is called *lateness-like* if there is a permutation, denoted $\mathbf{O} = (o(1), o(2), \dots, o(n))$, of the n tasks such that, for every time instant t , we have

$$f_{o(1)}(t) \geq f_{o(2)}(t) \geq \dots \geq f_{o(n)}(t). \quad (7)$$

From Eq. (7), it is not hard to see that the criteria $C_{\max}$, $WC_{\max}$, $L_{\max}$ and $T_{\max}$ are lateness-like criteria. In this paper, we only consider the lateness-like criteria $\tilde{L}_{\max} \in \{C_{\max}, WC_{\max}, L_{\max}, T_{\max}\}$. Then the permutation $\mathbf{O}$ for the criterion $\tilde{L}_{\max}$ with the property in (7) can be obtained in $O(n \log n)$ time. For example, if $\tilde{L}_{\max} = WC_{\max}$, then the permutation $\mathbf{O}$ is given by sorting the tasks such that $w_{o(1)} \geq w_{o(2)} \geq \dots \geq w_{o(n)}$.

2.2. Principle of Pareto optimization

Consider the Pareto scheduling problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$ for minimizing the total completion time $\sum C_j^{(A)}$ of the A -tasks and the maximum cost $f_{\max}^{(B)}$ of the B -tasks, simultaneously. A feasible schedule σ for problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$ is called *nondominated* if, for every feasible schedule π for this problem, the conditions $\sum C_j^{(A)}(\pi) \leq \sum C_j^{(A)}(\sigma)$ and $f_{\max}^{(B)}(\pi) \leq f_{\max}^{(B)}(\sigma)$ imply the relations $\sum C_j^{(A)}(\pi) = \sum C_j^{(A)}(\sigma)$ and $f_{\max}^{(B)}(\pi) = f_{\max}^{(B)}(\sigma)$. Equivalently, a nondominated schedule for problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$ is a feasible schedule σ such that there is no other feasible schedule π such that $(\sum C_j^{(A)}(\pi), f_{\max}^{(B)}(\pi)) < (\sum C_j^{(A)}(\sigma), f_{\max}^{(B)}(\sigma))$, where $(x, y) < (x', y')$ means that $x \leq x'$ and $y \leq y'$ and at least one of the two inequalities is strict. If σ is a nondominated schedule for problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$, we also say that $(\sum C_j^{(A)}(\sigma), f_{\max}^{(B)}(\sigma))$ is a *nondominated point* for this problem corresponding to the nondominated schedule σ . The goal of our Pareto scheduling problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$ is to find its all nondominated points, and their corresponding nondominated schedules. It should be noticed that, in the literature, nondominated schedule and nondominated point are also called Pareto-optimal schedule and Pareto-optimal point, respectively.

From the definition of nondominated point, or from the discussion in Hoogeveen [15] for general Pareto scheduling problems, we have the following lemma for our problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$.

Lemma 2.1. Consider problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)}, f_{\max}^{(B)})$ and let (x, y) be a vector of two real numbers. Then (x, y) is a nondominated point if and only if there exist an optimal schedule σ for problem $1|ND, k_j, p_j(t)|^{\#}(\sum C_j^{(A)} : f_{\max}^{(B)} \leq y)$ and an

optimal schedule σ' for problem $1|ND, k_j, p_j(t)|f_{\max}^{(B)} : \sum C_j^{(A)} \leq x$ such that $\sum C_j^{(A)}(\sigma) = x$ and $f_{\max}^{(B)}(\sigma') = y$. In this case, both σ and σ' are nondominated schedules corresponding to the same nondominated point (x, y) .

A feasible schedule σ for the constrained problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ is called a *nondominated-optimal schedule* if σ is an optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ and also a nondominated schedule for the Pareto scheduling problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$. Moreover, an algorithm for solving problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ is called a *nondominated-optimal algorithm* if the algorithm returns a *nondominated-optimal schedule* for this problem.

Note that the concepts of “nondominated-optimal schedule” and “nondominated-optimal algorithm” were first introduced in Geng and Yuan [13] in the terms of “optimal and Pareto-optimal schedule” and “optimal and Pareto-optimal algorithm”, respectively, for general Pareto scheduling problems. From the discussion in Geng and Yuan [13], we have the following method for solving the Pareto scheduling problem studied in this paper to generate all nondominated points $(C_1^{(A)}, F_1^{(B)}), (C_2^{(A)}, F_2^{(B)}), \dots, (C_N^{(A)}, F_N^{(B)})$ with $C_1^{(A)} < C_2^{(A)} < \dots < C_N^{(A)}$ (or equivalently, $F_1^{(B)} > F_2^{(B)} > \dots > F_N^{(B)}$) and their corresponding nondominated schedules $\pi_1, \pi_2, \dots, \pi_N$, respectively, where N is the number of nondominated points of problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$.

A Principle for Solving Problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$:

Suppose that, for every threshold value Q , we have a nondominated-optimal algorithm, denoted **Alg**(Q), for solving the constrained scheduling problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. Then the Pareto scheduling problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ can be solved by the following procedure:

- Initially set Q to be a sufficiently large number. Then we use the nondominated-optimal algorithm **Alg**(Q) for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. If this problem is feasible, then **Alg**(Q) returns a nondominated-optimal schedule π_1 . Set $(C_1^{(A)}, F_1^{(B)}) = (\sum C_j^{(A)}(\pi_1), f_{\max}^{(B)}(\pi_1))$. Then $(C_1^{(A)}, F_1^{(B)})$ is the first nondominated point for problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ with π_1 being a corresponding nondominated schedule.

- Generally, suppose that i nondominated points $(C_1^{(A)}, F_1^{(B)}), (C_2^{(A)}, F_2^{(B)}), \dots, (C_i^{(A)}, F_i^{(B)})$ with $C_1^{(A)} < C_2^{(A)} < \dots < C_i^{(A)}$ (or equivalently, $F_1^{(B)} > F_2^{(B)} > \dots > F_i^{(B)}$), together with their corresponding nondominated schedules $\pi_1, \pi_2, \dots, \pi_i$, have been generated. We reset $Q := F_i^{(B)} - 1$ and use the nondominated-optimal algorithm **Alg**(Q) for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. If this problem is infeasible, then we set $N := i$ and stop the procedure. If this problem is feasible, then **Alg**(Q) returns a nondominated-optimal schedule π_{i+1} . Set $(C_{i+1}^{(A)}, F_{i+1}^{(B)}) = (\sum C_j^{(A)}(\pi_{i+1}), f_{\max}^{(B)}(\pi_{i+1}))$. Then $(C_{i+1}^{(A)}, F_{i+1}^{(B)})$ is just the $(i+1)$ -st nondominated point for problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ with π_{i+1} being the corresponding nondominated schedule. After this, we repeat the above procedure until the final step. This completes the description of the principle for solving problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$.

As described in Geng and Yuan [13], the above discussion leads to the following lemma to estimate the time complexity for solving problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ by using nondominated-optimal algorithms.

Lemma 2.2. Suppose that the Pareto scheduling problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ has totally $N(n)$ nondominated points and, for each threshold value Q , we have a nondominated-optimal algorithm **Alg**(Q) for the constrained scheduling problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ with a time complexity $O(q(n))$ which has nothing to do with the choice of Q . Then problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ can be solved in $O(N(n)q(n))$ time.

3. Problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$

In this section, we consider the constrained ND-agent scheduling problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. For our purpose, we first provide the following definition:

Definition 3.1.

- (i) A task j is called a pure A-task, if j is in $\mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}$.
- (ii) A task j is called a pure B-task, if j is in $\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$.
- (iii) A task j is called an AB-task, if j is in $\mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$.

The following lemma establishes the properties which are used for finding the last task in an optimal schedule for the problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$.

Lemma 3.1. Suppose that the problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ is feasible and let $\tau = C(\mathcal{T}, \tau_0)$. Set

$$\mathcal{U}_A(\tau, |\mathcal{T}|) = \mathcal{U}_{A \setminus B}(|\mathcal{T}|) \cup \mathcal{U}_{AB}(\tau, |\mathcal{T}|),$$

where

$$\begin{cases} \mathcal{U}_{A \setminus B}(|\mathcal{T}|) = \{j \in \mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)} : k_j \geq |\mathcal{T}|\}, \\ \mathcal{U}_{AB}(\tau, |\mathcal{T}|) = \{j \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)} : f_j(\tau) \leq Q \text{ and } k_j \geq |\mathcal{T}|\}. \end{cases}$$

Then we have the following properties for problem 1|ND, k_j , $p_j(t)$ | $\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$:

- (i) If there is a pure B-task j such that $f_j^{(B)}(\tau) \leq Q$ and $k_j \geq |\mathcal{T}|$, then there is an optimal schedule in which j is the last task, and moreover, in every optimal schedule, the last task must be a pure B-task.
- (ii) If $f_i^{(B)}(\tau) > Q$ for each pure B-task i , then in any optimal schedule, the last task must be an A-task with the largest normal processing time in $\mathcal{U}_A(\tau, |\mathcal{T}|)$. Moreover, if j is a task in $\mathcal{U}_{A \setminus B}(|\mathcal{T}|)$ such that j has the largest normal processing time in $\mathcal{U}_A(\tau, |\mathcal{T}|)$, then there is an optimal schedule in which j is the last task. (This statement can be interpreted intuitively as that: If there are pure A-tasks and AB-tasks available at the last position and both of them have the largest normal processing time in $\mathcal{U}_A(\tau, |\mathcal{T}|)$, we make it a priority to choose a pure A-task.)

Proof. Suppose that σ is an optimal schedule for problem 1|ND, k_j , $p_j(t)$ | $\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. Suppose that j' is the last task in σ , i.e., $\sigma(|\mathcal{T}|) = j'$.

We first prove property (i).

Let j be a pure B-task such that $f_j^{(B)}(\tau) \leq Q$ and $k_j \geq |\mathcal{T}|$. If $\sigma(|\mathcal{T}|) = j$, then σ is a required optimal schedule such that the first statement of property (i) holds trivially. Suppose in the following that $\sigma(|\mathcal{T}|) = j'$ and $j' \neq j$. Then we have $k_{j'}, k_j \geq |\mathcal{T}|$ and $f_j^{(B)}(\tau) \leq Q$. Let σ' be the schedule obtained from σ by shifting j to the last position. It can be observed that $\sigma'[j] = |\mathcal{T}| \leq k_j$, $\sigma'[x] \leq \sigma[x] \leq k_x$ and $C_x(\sigma') \leq C_x(\sigma)$ for any task x with $x \neq j$. It follows that $f_{x'}^{(B)}(C_{x'}(\sigma')) \leq f_{x'}^{(B)}(C_{x'}(\sigma)) \leq Q$ for any B-task x' with $x' \neq j$. Moreover, from Eq. (1), we have that $C_j(\sigma') = C_{j'}(\sigma) = \tau$.

In the case that j' is an A-task, we have $C_{j'}(\sigma') < C_{j'}(\sigma)$, and so, $\sum C_j^{(A)}(\sigma') < \sum C_j^{(A)}(\sigma)$. This contradicts the optimality of σ . Hence, the task j' must be a pure B-task. Since $C_j(\sigma') = C_{j'}(\sigma) = \tau$, we have $f_j^{(B)}(C_j(\sigma')) = f_j^{(B)}(\tau) \leq Q$. This means that $f_{\max}^{(B)}(\sigma') \leq Q$ and $\sum C_j^{(A)}(\sigma') \leq \sum C_j^{(A)}(\sigma)$. Since j is the last task in σ' , we conclude that σ' is a required optimal schedule. This proves property (i).

Now we prove property (ii). It is noticed that $f_i^{(B)}(\tau) > Q$ for each pure B-task i . Then the last task, say j' , in σ (in fact, in every optimal schedule) must be an A-task. Hence, we have $j' \in \mathcal{U}_A(\tau, |\mathcal{T}|)$. Let $\alpha_{\max}^{(A)}$ be the largest normal processing time of the tasks in $\mathcal{U}_A(\tau, |\mathcal{T}|)$.

By contradiction, we assume that $\alpha_{j'} < \alpha_{\max}^{(A)}$. Then there is a task $j \in \mathcal{U}_A(\tau, |\mathcal{T}|)$ such that $\alpha_{j'} < \alpha_j \leq \alpha_{\max}^{(A)}$ and $|\mathcal{T}| \leq \min\{k_{j'}, k_j\}$. Moreover, if $x \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$, then $f_x^{(B)}(\tau) \leq Q$ for $x \in \{j', j\}$. Let σ'' be the schedule obtained from σ by exchanging the positions of tasks j' and j . Then we have $\sigma''(|\mathcal{T}|) = j$. Clearly, $\sigma''[l] \leq \sigma[l] \leq k_l$ for every task l with $l \neq j$. Moreover, from the fact $\alpha_{j'} < \alpha_j$ and from Eq. (1), we have $C_l(\sigma'') \leq C_l(\sigma)$ for $l \notin \{j', j\}$, $C_{j'}(\sigma'') \leq C_j(\sigma) < \tau = C_{j'}(\sigma)$ and $C_j(\sigma'') = C_{j'}(\sigma) = \tau$. It follows that $f_l^{(B)}(C_l(\sigma'')) \leq f_l^{(B)}(C_l(\sigma)) \leq Q$ for any B-task l with $l \neq j$ and $\sum C_j^{(A)}(\sigma'') \leq \sum C_j^{(A)}(\sigma)$. The choice of j also implies that $f_j^{(B)}(C_j(\sigma'')) \leq Q$ if j is an AB-task. Thus, $f_{\max}^{(B)}(\sigma'') \leq Q$ and $\sum C_j^{(A)}(\sigma'') \leq \sum C_j^{(A)}(\sigma)$. Consequently, σ'' is a required optimal schedule. The lemma follows: $\square$

From Eq. (1), when the last task j is determined, the updated value $C(\mathcal{T} \setminus \{j\}, \tau_0)$ can be calculated in a constant time. Based on Lemma 3.1, we present the following algorithm for problem 1|ND, k_j , $p_j(t)$ | $\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. The feasibility of the problem is checked in the implementation of the algorithm.

Algorithm 1. For problem 1|ND, k_j , $p_j(t)$ | $\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$.

Step 1: Set $\tau := C(\mathcal{T}, \tau_0)$ and $l := n$.

Step 2: If $\tau = \tau_0$, then stop the algorithm.

Step 3: Determine the task sets

$$\begin{cases} \mathcal{U}_{B \setminus A}(\tau, l) &= \{j \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)} : f_j(\tau) \leq Q \text{ and } k_j \geq l\}, \\ \mathcal{U}_{A \setminus B}(l) &= \{j \in \mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)} : k_j \geq l\}, \\ \mathcal{U}_{AB}(\tau, l) &= \{j \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)} : f_j(\tau) \leq Q \text{ and } k_j \geq l\}, \\ \mathcal{U}_A(\tau, l) &= \mathcal{U}_{A \setminus B}(l) \cup \mathcal{U}_{AB}(\tau, l). \end{cases}$$

If $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \emptyset$, then the problem is infeasible and stop the algorithm.

Step 4: If $\mathcal{U}_{B \setminus A}(\tau, l) \neq \emptyset$, then determine a task $i \in \mathcal{U}_{B \setminus A}(\tau, l)$ with the smallest cost at time τ . If there is a tie, we choose the task with the largest normal processing time. Schedule task i as the l -th task. Set $l := l - 1$, $\mathcal{T} := \mathcal{T} \setminus \{i\}$ and $\tau := C(\mathcal{T}, \tau_0)$. Return to Step 2.

Step 5: If $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) \neq \emptyset$, then choose a task $j \in \mathcal{U}_A(\tau, l)$ with the largest normal processing time. If there is a tie, we make it a priority to choose a pure A-task. Moreover, if $\mathcal{U}_{A \setminus B}(l) = \emptyset$ and $|\mathcal{U}_{AB}(\tau, l)| \geq 2$, we choose a AB-task

Table 2
The data of instance $\mathcal{I}$.

j	1	2	3	4	5	6	7	8	9
α_j	1	1	2	1	2	1	1	2	1
k_j	9	8	6	7	6	7	5	9	9
$f_j(x)$			$4x + 10$		$6x + 8$	$5x + 9$	$8x + 5$	$x + 2$	$x + 7$

6	4	2	5	7	3	1	9	8	
τ_0	2	4	8	24	48	144	288	576	1728

Fig. 1. The schedule of instance $\mathcal{I}$ obtained by Algorithm 1.

with the smallest cost at time τ . Schedule task j as the l -th task. Set $l := l - 1$ and $\mathcal{T} := \mathcal{T} \setminus \{j\}$ and $\tau := C(\mathcal{T}, \tau_0)$. Return to Step 2.

Now we use an instance $\mathcal{I}$ to show how Algorithm 1 works. Set $a = 0$, $b = 1$, $\tau_0 = 1$ and $Q = 1735$. Moreover, let $n = 9$, $\mathcal{T}^{(A)} := \{1, 2, 3, 4, 5, 6\}$ and $\mathcal{T}^{(B)} := \{3, 5, 6, 7, 8, 9\}$. Note that tasks 1, 2 and 4 are pure A-tasks, tasks 3, 5 and 6 are AB-tasks, and tasks 7, 8 and 9 are pure B-tasks. We define the data of instance $\mathcal{I}$ in Table 2.

Algorithm 1 on instance $\mathcal{I}$ works in the following way:

At the iteration $l = 9$, we have $\tau = 1728$, $\mathcal{U}_{B \setminus A}(\tau, l) = \{8, 9\}$. Moreover, $f_8(\tau) < f_9(\tau) = Q$. From Algorithm 1, the pure B-task 8 is scheduled as the ninth task completing at time 1728.

At the iteration $l = 8$, we have $\tau = 576$ and $\mathcal{U}_{B \setminus A}(\tau, l) = \{9\}$. From Algorithm 1, the pure B-task 9 is scheduled as the eighth task completing at time 576.

At the iteration $l = 7$, we have $\tau = 288$, $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \{1, 2, 4, 6\}$, where $\mathcal{U}_{A \setminus B}(\tau, l) = \{1, 2, 4\}$ and $\mathcal{U}_{AB}(\tau, l) = \{6\}$. Note that $\alpha_1 = \alpha_2 = \alpha_4 = \alpha_6 = 1$. From Algorithm 1, all the three pure A-tasks 1, 2 and 4 can be scheduled as the seventh task. Since any choice guarantees the optimality, we assume that the pure A-task 1 is scheduled as the seventh task completing at time 288.

At the iteration $l = 6$, we have $\tau = 144$, $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \{2, 3, 4, 5, 6\}$, where $\mathcal{U}_{A \setminus B}(\tau, l) = \{2, 4\}$ and $\mathcal{U}_{AB}(\tau, l) = \{3, 5, 6\}$. The two AB-tasks 3 and 5 have the maximum normal processing time among all the A-tasks in $\mathcal{U}_A(\tau, l)$. Moreover, $f_3(\tau) < f_5(\tau) < Q$. From Algorithm 1, the AB-task 3 is scheduled as the sixth task completing at time 144.

At the iteration $l = 5$, we have $\tau = 48$ and $\mathcal{U}_{B \setminus A}(\tau, l) = \{7\}$. From Algorithm 1, the pure B-task 7 is scheduled as the fifth task completing at time 48.

At the iteration $l = 4$, we have $\tau = 24$, $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \{2, 4, 5, 6\}$, where $\mathcal{U}_{A \setminus B}(\tau, l) = \{2, 4\}$ and $\mathcal{U}_{AB}(\tau, l) = \{5, 6\}$. Note that $\alpha_2 = \alpha_4 = \alpha_6 = 1$ and $\alpha_5 = 2$. From Algorithm 1, the AB-task 5 is scheduled as the fourth task completing at time 24.

At the iteration $l = 3$, we have $\tau = 8$, $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \{2, 4, 6\}$, where $\mathcal{U}_{A \setminus B}(\tau, l) = \{2, 4\}$ and $\mathcal{U}_{AB}(\tau, l) = \{6\}$. Note that $\alpha_2 = \alpha_4 = \alpha_6 = 1$. From Algorithm 1, both the pure A-tasks 2 and 4 can be scheduled as the third task. We assume that the pure A-task 2 is scheduled as the third task completing at time 8.

At the iteration $l = 2$, we have $\tau = 4$, $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \{4, 6\}$, where $\mathcal{U}_{A \setminus B}(\tau, l) = \{4\}$ and $\mathcal{U}_{AB}(\tau, l) = \{6\}$. Note that $\alpha_4 = \alpha_6 = 1$. From Algorithm 1, the pure A-task 4 is scheduled as the second task completing at time 4.

At the iteration $l = 1$, we have $\tau = 2$, $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \mathcal{U}_{AB}(\tau, l) = \{6\}$. From Algorithm 1, the AB-task 6 is scheduled as the first task completing at time 2.

The schedule obtained by Algorithm 1 on instance $\mathcal{I}$ is displayed in Fig. 1.

Lemma 3.2. The running time of Algorithm 1 is $O(n^2)$.

Proof. Note that it takes $O(n)$ time to calculate $\mathcal{U}_{B \setminus A}(\tau, l)$ and $\mathcal{U}_A(\tau, l)$ at each iteration. If $\mathcal{U}_{B \setminus A}(\tau, l) \neq \emptyset$, a pure B-task i can be determined in $O(|\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}|) = O(n)$ time at Step 4. If $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) \neq \emptyset$, an A-task j will be determined in $O(|\mathcal{T}^{(A)}|) = O(n)$ time at Step 5. Therefore, each iteration of Algorithm 1 runs in $O(n)$ time. Since we have totally n iterations, the overall time complexity of Algorithm 1 is given by $O(n^2)$. $\square$

Lemma 3.3. Suppose that the problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ is feasible. Then Algorithm 1 returns a nondominated-optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. Consequently, Algorithm 1 is a nondominated-optimal algorithm for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$.

Proof. Let σ be the schedule obtained by Algorithm 1. From Lemma 3.1, σ is optimal for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. Since $f_{\max}^{(B)}(\sigma) \leq Q$, σ is also optimal for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq f_{\max}^{(B)}(\sigma)$. From Lemma 2.1, we need to show that σ is also an optimal schedule for problem $1|ND, k_j, p_j(t)|f_{\max}^{(B)} : \sum C_j^{(A)} \leq \sum C_j^{(A)}(\sigma)$.

Suppose that π is an optimal schedule for problem $1|ND, k_j, p_j(t)|f_{\max}^{(B)} : \sum C_j^{(A)} \leq \sum C_j^{(A)}(\sigma)$. Then we have that $f_{\max}^{(B)}(\pi) \leq f_{\max}^{(B)}(\sigma)$ and $\sum C_j^{(A)}(\pi) \leq \sum C_j^{(A)}(\sigma)$. This implies that π is optimal for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. Hence, we have $\sum C_j^{(A)}(\pi) = \sum C_j^{(A)}(\sigma)$, and from Lemma 2.1, we know that π is a nondominated-optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. Thus, we only need to show that $f_{\max}^{(B)}(\pi) = f_{\max}^{(B)}(\sigma)$.

Suppose to the contrary that $f_{\max}^{(B)}(\pi) < f_{\max}^{(B)}(\sigma)$. Then π and σ are two different schedules. For convenience, we assume in the sequel that the schedule π is chosen such that the index k_π is as small as possible, where k_π is the maximum positional index with $\pi(k_\pi) \neq \sigma(k_\pi)$. The choice of the index k_π implies that $\pi(i) = \sigma(i)$ for $i = k_\pi, k_\pi + 1, \dots, n$. Consequently, the set of the first k_π tasks in schedule π is the same as that in schedule σ . Let $u = \pi(k_\pi)$ and $v = \sigma(k_\pi)$, which are the k_π -th tasks in π and in σ , respectively. From equation (1), we have $C_u(\pi) = C_v(\sigma)$ by considering the first k_π tasks in σ and π . Set $\tau = C_u(\pi) = C_v(\sigma)$.

If $u, v \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$, from Algorithm 1, we have $k_\pi \leq \min\{k_u, k_v\}$ and $f_v^{(B)}(\tau) \leq f_u^{(B)}(\tau) \leq Q$. Let π' be the schedule obtained from π by shifting task v to the k_π th position. Then we have

$$\begin{cases} \pi'[v] = k_\pi, \\ \pi'[x] \leq \pi[x] \leq k_x, & \text{for } x \in \mathcal{T} \setminus \{v\}, \\ C_x(\pi') \leq C_x(\pi), & \text{for } x \in \mathcal{T} \setminus \{v\}. \end{cases}$$

Consequently, we have $f_x^{(B)}(C_x(\pi')) \leq f_x^{(B)}(C_x(\pi)) \leq f_{\max}^{(B)}(\pi)$ for any B -task $x \in \mathcal{T}^{(B)} \setminus \{v\}$. From Eq. (1), we further have that $C_v(\pi') = C_u(\pi) = \tau$, and so, $f_v^{(B)}(C_v(\pi')) = f_v^{(B)}(\tau) \leq f_u^{(B)}(\tau) = f_u^{(B)}(C_u(\pi)) \leq f_{\max}^{(B)}(\pi)$. It follows that $f_{\max}^{(B)}(\pi') \leq f_{\max}^{(B)}(\pi)$ and $\sum C_j^{(A)}(\pi') \leq \sum C_j^{(A)}(\pi)$. Since π is a nondominated-optimal schedule, we must have $f_{\max}^{(B)}(\pi') = f_{\max}^{(B)}(\pi)$ and $\sum C_j^{(A)}(\pi') = \sum C_j^{(A)}(\pi)$. Consequently, π' is also a nondominated-optimal schedule. If $\pi' = \sigma$, then we have $f_{\max}^{(B)}(\sigma) = f_{\max}^{(B)}(\pi') = f_{\max}^{(B)}(\pi)$, contradicting the hypothesis that $f_{\max}^{(B)}(\pi) < f_{\max}^{(B)}(\sigma)$. If $\pi' \neq \sigma$, then we have $k_{\pi'} < k_\pi$, contradicting the choice of π . Thus, we always have a contradiction.

If $u \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$ and $v \in \mathcal{T}^{(A)}$, then we notice that $\sigma|_{k_\pi}$ is an optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on the first k_π tasks in σ and u is a pure B -task with $f_u^{(B)}(C_u(\sigma)) \leq Q$. By Lemma 3.1, the A -task v cannot be processed as the k_π -th task in any optimal schedule. This leads to a contradiction.

If $u \in \mathcal{T}^{(A)}$ and $v \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$, then we notice that $\pi|_{k_\pi}$ is an optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on the first k_π tasks in π and v is a pure B -task with $f_v^{(B)}(C_v(\pi)) \leq Q$. From Lemma 3.1, the A -task u cannot be processed as the k_π -th task in any optimal schedule. This leads to a contradiction.

From the above discussion, we may assume in the following that both u and v are tasks in $\mathcal{T}^{(A)}$. Then we notice that $k_\pi \leq \min\{k_u, k_v\}$ and both $\pi|_{k_\pi}$ and $\sigma|_{k_\pi}$ are optimal for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on the first k_π tasks in π (and also in σ). From Lemma 3.1, we have $\alpha_u = \alpha_v$. Let π'' be the schedule obtained from π by exchanging the positions of u and v . Clearly, $\pi''[x] \leq \pi[x] \leq k_x$ for every task x with $x \neq v$, and $\pi''[v] = k_\pi \leq k_v$. Moreover, from the fact $\alpha_u = \alpha_v$ and from Eq. (1), we have $C_x(\pi'') = C_x(\pi)$ for $x \notin \{u, v\}$, $C_u(\pi'') = C_v(\pi) \leq \tau$ and $C_v(\pi'') = C_u(\pi) = \tau$. It follows that $f_x^{(B)}(C_x(\pi'')) \leq f_x^{(B)}(C_x(\pi)) \leq Q$ for any B -task $x \in \mathcal{T}^{(B)} \setminus \{u, v\}$, and $\sum C_j^{(A)}(\pi'') = \sum C_j^{(A)}(\pi)$. In the following, we show that $f_{\max}^{(B)}(\pi'') \leq f_{\max}^{(B)}(\pi)$.

If $u, v \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$, from Algorithm 1, we have $f_v^{(B)}(\tau) \leq f_u^{(B)}(\tau)$. Since $C_u(\pi'') \leq C_u(\pi)$ and $C_v(\pi'') = C_u(\pi) = \tau$, we have $f_u^{(B)}(C_u(\pi'')) \leq f_u^{(B)}(C_u(\pi))$ and $f_v^{(B)}(C_v(\pi'')) = f_v^{(B)}(\tau) \leq f_u^{(B)}(\tau) = f_u^{(B)}(C_u(\pi))$. It follows that $f_{\max}^{(B)}(\pi'') \leq f_{\max}^{(B)}(\pi)$, as required.

If $u \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$ and $v \in \mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}$, then we notice that $\pi|_{k_\pi}$ is an optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on the first k_π tasks in π and v is a pure A -task with $k_\pi \leq k_v$. From Lemma 3.1, the AB -task u cannot be processed as the k_π th task in any optimal schedule, a contradiction. Thus, this case cannot occur.

If $u \in \mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}$ and $v \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$, then we notice that $\sigma|_{k_\pi}$ is an optimal schedule for problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on the first k_π tasks in σ and u is a pure A -task with $k_\pi \leq k_u$. By Lemma 3.1, the AB -task v cannot be processed as the k_π th task in any optimal schedule, a contradiction. Thus, this case cannot occur.

If $u, v \in \mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}$, then we clearly have $f_{\max}^{(B)}(\pi'') = f_{\max}^{(B)}(\pi)$, as required.

The above discussion shows that $f_{\max}^{(B)}(\pi'') \leq f_{\max}^{(B)}(\pi)$ and $\sum C_j^{(A)}(\pi'') = \sum C_j^{(A)}(\pi)$. Since π is a nondominated-optimal schedule, we certainly have $f_{\max}^{(B)}(\pi'') = f_{\max}^{(B)}(\pi)$ and $\sum C_j^{(A)}(\pi'') = \sum C_j^{(A)}(\pi)$. Consequently, π'' is also a nondominated-optimal schedule. If $\pi'' = \sigma$, then we have $f_{\max}^{(B)}(\sigma) = f_{\max}^{(B)}(\pi'') = f_{\max}^{(B)}(\pi)$, contradicting the hypothesis that $f_{\max}^{(B)}(\pi) < f_{\max}^{(B)}(\sigma)$. If $\pi'' \neq \sigma$, then we have $k_{\pi''} < k_\pi$, contradicting the choice of π . The lemma follows: $\square$

From Lemmas 3.2 and 3.3, we conclude the following result:

Theorem 3.1. Problem $1|ND, k_j, p_j(t)|\sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ is solvable in $O(n^2)$ time.

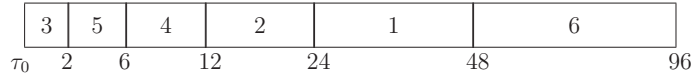


Fig. 2. The schedule of instance $\tilde{\mathcal{I}}$ obtained by Algorithm 3.

Now we consider problem $1|ND, p_j(t)| \sum C_j^{(A)} : \tilde{L}_{\max}^{(B)} \leq Q$ without positional constraint. Based on Lemma 3.1, we present the following algorithm for problem $1|ND, p_j(t)| \sum C_j^{(A)} : \tilde{L}_{\max}^{(B)} \leq Q$. Recall that $\tilde{L}_{\max}^{(B)} \in \{C_{\max}^{(B)}, WC_{\max}^{(B)}, L_{\max}^{(B)}, T_{\max}^{(B)}\}$ is a lateness-like criterion.

Algorithm. $\tilde{\mathcal{I}}$ For problem $1|ND, p_j(t)| \sum C_j^{(A)} : \tilde{L}_{\max}^{(B)} \leq Q$.

Step 1: Set $\tau := C(\mathcal{T}, \tau_0)$ and $l := n$.

Step 2: If $\tau = \tau_0$, then stop the algorithm.

Step 3: Determine the task sets

$$\begin{cases} \mathcal{U}_{B \setminus A}(\tau) = \{j \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)} : \tilde{L}_j(\tau) \leq Q\}, \\ \mathcal{U}_{AB}(\tau) = \{j \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)} : \tilde{L}_j(\tau) \leq Q\}, \\ \mathcal{U}_A(\tau) = \mathcal{U}_{AB}(\tau) \cup (\mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}). \end{cases}$$

If $\mathcal{U}_{B \setminus A}(\tau) = \emptyset$ and $\mathcal{U}_A(\tau) = \emptyset$, then the problem is infeasible and stop the algorithm.

Step 4: If $\mathcal{U}_{B \setminus A}(\tau) \neq \emptyset$, then choose a task $i \in \mathcal{U}_{B \setminus A}(\tau)$ such that $\tilde{L}_i(\tau) = \min_{j \in \mathcal{U}_{B \setminus A}(\tau)} \tilde{L}_j(\tau)$. If there is a tie, choose one with the largest normal processing time. Schedule task i as the l th task. Reset $l := l - 1$, $\mathcal{T} := \mathcal{T} \setminus \{i\}$ and $\tau := C(\mathcal{T}, \tau_0)$. Return to Step 2.

Step 5: If $\mathcal{U}_{B \setminus A}(\tau) = \emptyset$ and $\mathcal{U}_A(\tau) \neq \emptyset$, then choose a task $j \in \mathcal{U}_A(\tau)$ with the largest normal processing time. If there is a tie, we make it a priority to choose a pure A-task. Moreover, if $\mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)} = \emptyset$ and $|\mathcal{U}_{AB}(\tau)| \geq 2$, we choose a AB-task with the smallest objective value at time τ . Schedule task j as the l -th task. Set $l := l - 1$ and $\mathcal{T} := \mathcal{T} \setminus \{j\}$ and $\tau := C(\mathcal{T}, \tau_0)$. Return to Step 2.

Now we use an instance $\tilde{\mathcal{I}}$ to show how Algorithm 3 works. Set $a = 0$, $b = 1$, $\tau_0 = 1$, $Q = 8$, and $\tilde{L}_{\max}^{(B)} = L_{\max}^{(B)}$. Moreover, let $n = 6$, $\mathcal{T}^{(A)} := \{1, 2, 3, 4\}$ and $\mathcal{T}^{(B)} := \{3, 4, 5, 6\}$. Note that tasks 1 and 2 are pure A-tasks, tasks 3 and 4 are AB-tasks, and tasks 5 and 6 are pure B-tasks. We use $(j; \alpha_j)$ to denote the α_j value related to pure A-task j , and use $(j; \alpha_j, d_j)$ to denote the α_j and d_j values related to B-task j . Then we define the data of instance $\tilde{\mathcal{I}}$ as: (1; 1), (2; 1), (3; 1, 6), (4; 1, 40), (5; 2, 3) and (6; 1, 90).

Algorithm 3 on instance $\tilde{\mathcal{I}}$ works as follows:

At the iteration $l = 6$, we have $\tau = 96$ and $\mathcal{U}_{B \setminus A}(\tau) = \{6\}$. From Algorithm 3, the pure B-task 6 is scheduled as the sixth task completing at time 96.

At the iteration $l = 5$, we have $\tau = 48$, $\mathcal{U}_{B \setminus A}(\tau) = \emptyset$ and $\mathcal{U}_A(\tau) = \{1, 2, 4\}$, where 1 and 2 are pure A-tasks and 4 is an AB-task. Note that $\alpha_1 = \alpha_2 = \alpha_4 = 1$. From Algorithm 3, both the pure A-tasks 1 and 2 can be scheduled as the fifth task. Since any choice guarantees the optimality, we suppose that the pure A-task 1 is scheduled as the fifth task completing at time 48.

At the iteration $l = 4$, we have $\tau = 24$, $\mathcal{U}_{B \setminus A}(\tau) = \emptyset$ and $\mathcal{U}_A(\tau) = \{2, 4\}$, where 2 is a pure A-task and 4 is an AB-task. Note that $\alpha_2 = \alpha_4 = 1$. From Algorithm 3, the pure A-task 2 is scheduled as the fourth task completing at time 24.

At the iteration $l = 3$, we have $\tau = 12$, $\mathcal{U}_{B \setminus A}(\tau) = \emptyset$ and $\mathcal{U}_A(\tau) = \{3, 4\}$, where both 3 and 4 are AB-tasks. Note that $\alpha_3 = \alpha_4 = 1$ and $d_3 < d_4$. From Algorithm 3, the AB-task 4 is scheduled as the third task completing at time 12.

At the iteration $l = 2$, we have $\tau = 6$ and $\mathcal{U}_{B \setminus A}(\tau) = \{5\}$. From Algorithm 3, the pure B-task 5 is scheduled as the second task completing at time 6.

At the iteration $l = 1$, we have $\tau = 2$, $\mathcal{U}_{B \setminus A}(\tau) = \emptyset$ and $\mathcal{U}_A(\tau) = \{3\}$. From Algorithm 3, the AB-task 3 is scheduled as the first task completing at time 2.

The schedule obtained by Algorithm 3 on instance $\tilde{\mathcal{I}}$ is displayed in Fig. 2.

Lemma 3.4. The running time of Algorithm 3 is $O(n \log n)$.

Proof. The time complexity is discussed by using the technique of data structure. In the preprocessing procedure, we first generate three lists acting as the data structure:

- $\mathcal{L}_{B-A}$ is the list which sorts the tasks in $\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$ in the order described in (7).
- $\mathcal{L}_{AB}$ is the list which sorts the tasks in $\mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$ in the order described in (7).
- $\mathcal{L}_A$ is the list which sorts the tasks in $\mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}$ in the nondecreasing order of normal processing times.

Note that the three lists can be generated in $O(n \log n)$ time by the sorting algorithm. Then we update these lists in the implementation of the algorithm iteratively. The algorithm begins at time $\tau = C(\mathcal{T}, \tau_0)$. At the beginning of each iteration, the three lists have the following meanings:

– $\mathcal{L}_{B-A}$ stores the unscheduled pure B -tasks in $\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$. From (7), if any task in $\mathcal{L}_{B-A}$ is available for being chosen at time τ , then the last task in list $\mathcal{L}_{B-A}$ must be available for being chosen at time τ .

– $\mathcal{L}_{AB} \cup \mathcal{L}_A$ stores the unscheduled A -tasks, where $\mathcal{L}_A$ consists of all unscheduled pure A -tasks in $\mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)}$ and some unscheduled AB -tasks in $\mathcal{T}^{(A)} \cap \mathcal{T}^{(B)}$ which are available for being chosen at time τ , and $\mathcal{L}_{AB}$ consists of other unscheduled AB -tasks not included in $\mathcal{L}_A$. Thus, all tasks in $\mathcal{L}_A$ are available for being chosen at time τ , and, from (7), if any task in $\mathcal{L}_{AB}$ is available for being chosen at time τ , then the last task in list $\mathcal{L}_{AB}$ must be available for being chosen at time τ .

At the implementation of the iteration for scheduling at time τ , we do the following:

- (i) If $\mathcal{L}_{B-A} \neq \emptyset$ and the last task in $\mathcal{L}_{B-A}$, denoted j , is available for being chosen at time τ , then we delete j from $\mathcal{L}_{B-A}$ and schedule j to complete at time τ . This procedure will be repeated at the next iteration until either $\mathcal{L}_{B-A} = \emptyset$ or the last task in $\mathcal{L}_{B-A}$ is no longer available for being chosen at the updated time τ .
- (ii) If $\mathcal{L}_{AB} \neq \emptyset$ and the last task in $\mathcal{L}_{AB}$, denoted j , is available for being chosen at time τ , then we delete j from $\mathcal{L}_{AB}$ and insert j into the list $\mathcal{L}_A$ such that the nondecreasing order of normal processing times of the tasks in $\mathcal{L}_A$ unchanged. This procedure is repeated at the current iteration until either $\mathcal{L}_{AB} = \emptyset$ or the last task in $\mathcal{L}_{AB}$ is no longer available for being chosen at time τ . After such implementations, $\mathcal{L}_A$ consists of all A -tasks available for being chosen at time τ , i.e., $\mathcal{L}_A$ is a sorting of the tasks in $U_A(\tau)$.
- (iii) If $\mathcal{L}_A \neq \emptyset$, we obtain the last task in $\mathcal{L}_A$, denoted j . Then we delete j from $\mathcal{L}_A$ and schedule j to complete at time τ . After this, we enter the next iteration.

Each operation for inserting a new task to list $\mathcal{L}_A$ or deleting the last task in lists $\mathcal{L}_{B-A}$, $\mathcal{L}_{AB}$ or $\mathcal{L}_A$ takes $O(\log n)$ time by binary-search. Moreover, scheduling a task in each iteration needs a constant time. Hence, the total time complexity for generating and updating the three lists and scheduling all the tasks is bounded by $O(n \log n)$ time. Consequently, the overall time complexity of Algorithm 3 is given by $O(n \log n)$. The lemma follows. $\square$

As a consequence of Lemma 3.3, we have the following lemma.

Lemma 3.5. Algorithm 3 generates a nondominated-optimal schedule for the problem $1|ND, p_j(t)| \sum C_j^{(A)} : \tilde{L}_{\max}^{(B)} \leq Q$, where $\tilde{L}_{\max}^{(B)} \in \{C_{\max}^{(B)}, W_{\max}^{(B)}, L_{\max}^{(B)}, T_{\max}^{(B)}\}$.

From Lemmas 3.4 and 3.5, we conclude the following result:

Theorem 3.2. Problem $1|ND, p_j(t)| \sum C_j^{(A)} : \tilde{L}_{\max}^{(B)} \leq Q$ is solvable in $O(n \log n)$ time, where $\tilde{L}_{\max}^{(B)} \in \{C_{\max}^{(B)}, W_{\max}^{(B)}, L_{\max}^{(B)}, T_{\max}^{(B)}\}$.

4. Problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$

In this section, we consider the Pareto ND-agent scheduling problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$. From Lemma 3.3, Algorithm 1 is a nondominated-optimal algorithm for the corresponding constrained scheduling problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$. From Lemma 2.2, we need to estimate the number of nondominated points.

Suppose that $|\mathcal{T}^{(A)}| = n_A$, $\mathcal{T}^{(A)} = \{1, 2, \dots, n_A\}$ and $\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)} = \{n_A + 1, \dots, n\}$. Without loss of generality, we assume in the sequel that $\alpha_1 \leq \alpha_2 \leq \dots \leq \alpha_{n_A}$. Moreover, we set $\bar{\mathcal{T}}_A = (1, 2, \dots, n_A)$, where if an AB -task u and a pure A -task v have the same normal processing time, then $u < v$ in $\bar{\mathcal{T}}_A$. For a given schedule σ of $\mathcal{T}$, we define

$$\lambda_1(\sigma) = |\{(i, j) : 1 \leq i < j \leq n_A, \sigma[j] < \sigma[i]\}|, \quad (8)$$

and

$$\lambda_2(\sigma) = |\{(i, j) : 1 \leq i \leq n_A, n_A + 1 \leq j \leq n, \sigma[j] < \sigma[i]\}|. \quad (9)$$

From Eqs. (8) and (9), it is easy to verify that $\lambda_1(\sigma) \leq \frac{n_A(n_A-1)}{2}$ and $\lambda_2(\sigma) \leq n_A(n - n_A)$. Thus we have

$$\lambda_1(\sigma) + \lambda_2(\sigma) \leq \frac{n_A(n_A-1)}{2} + n_A(n - n_A). \quad (10)$$

In order to estimate the number of nondominated points, we introduce the following auxiliary algorithm to solve problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$.

Algorithm 2. For problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$.

Step 1: Set $\tau := C(\mathcal{T}, \tau_0)$ and $l := n$.

Step 2: If $\tau = \tau_0$, then stop.

Step 3: Determine the task sets

$$\begin{cases} \mathcal{U}_{B \setminus A}(\tau, l) = \{j \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)} : f_j(\tau, l) \leq Q \text{ and } k_j \geq l\}, \\ \mathcal{U}_{A \setminus B}(l) = \{j \in \mathcal{T}^{(A)} \setminus \mathcal{T}^{(B)} : k_j \geq l\}, \\ \mathcal{U}_{AB}(\tau, l) = \{j \in \mathcal{T}^{(A)} \cap \mathcal{T}^{(B)} : f_j(\tau) \leq Q \text{ and } k_j \geq l\}, \\ \mathcal{U}_A(\tau, l) = \mathcal{U}_{A \setminus B}(l) \cup \mathcal{U}_{AB}(\tau, l). \end{cases}$$

If $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) = \emptyset$, then the problem is infeasible and stop the algorithm.

Step 4: If $\mathcal{U}_{B \setminus A}(\tau, l) \neq \emptyset$, then choose an arbitrary task $i \in \mathcal{U}_{B \setminus A}(\tau, l)$, and schedule task i as the l -th task. Set $l := l - 1$, $\mathcal{T} := \mathcal{T} \setminus \{i\}$ and $\tau := C(\mathcal{T}, \tau_0)$. Return to Step 2.

Step 5: If $\mathcal{U}_{B \setminus A}(\tau, l) = \emptyset$ and $\mathcal{U}_A(\tau, l) \neq \emptyset$, then we find the task $j = \max\{x : x \in \mathcal{U}_A(\tau, l)\}$, i.e., j is the last available A -task for being chosen in $\mathcal{T}_A$. Schedule task j as the l -th task. Set $l := l - 1$, $\mathcal{T} := \mathcal{T} \setminus \{j\}$ and $\tau := C(\mathcal{T}, \tau_0)$. Return to Step 2.

Remark 1. Note that the difference between Algorithm 1 and Algorithm 2 is just at Step 5 of the two algorithms. At the Step 5 of Algorithm 1, if there is a tie, we can choose an arbitrary available A -task to be scheduled at the last position. Then the optimal schedule obtained by Algorithm 1 may be not unique. In Algorithm 2, the A -tasks are relabeled in $\mathcal{T}_A$ in advance. This guarantee that, at Step 5 of Algorithm 2, there is only one choice for available A -tasks to be scheduled at the last position. Then the optimal schedule obtained by Algorithm 2 is unique. The introducing of Algorithm 2 is just to estimate the number of nondominated points.

Lemma 4.1. Suppose that problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ is feasible and let π be the schedule generated by Algorithm 2. Then we have the following three statements:

- (i) π is an optimal schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$.
- (ii) For every feasible schedule σ for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$, we have $\lambda_1(\pi) + \lambda_2(\pi) \leq \lambda_1(\sigma) + \lambda_2(\sigma)$.
- (iii) For every feasible schedule σ for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ with $\sum C_j^{(A)}(\sigma) > \sum C_j^{(A)}(\pi)$, we have $\lambda_1(\pi) + \lambda_2(\pi) < \lambda_1(\sigma) + \lambda_2(\sigma)$.

Proof. From Lemma 3.1 and from the implementation of Algorithm 2, statement (i) follows obviously. We then prove statements (ii) and (iii) in the following.

For each x with $1 \leq x \leq n$, we define $\mathcal{I}_x = \{\pi(1), \pi(2), \dots, \pi(x)\}$ to be the task instance consisting of the first x tasks in π . Note that $\pi|_x$ is the schedule obtained by Algorithm 2 on instance $\mathcal{I}_x$. It follows that $\pi|_x$ is optimal for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_x$. Then we only need to show that, for each x with $1 \leq x \leq n$ and for each feasible schedule σ_x for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_x$, statements (ii) and (iii) hold for $\pi|_x$ and σ_x .

Suppose that σ_x is a feasible schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_x$ such that one of statements (ii) and (iii) is violated for $\pi|_x$ and σ_x . For our purpose, we may assume that such an index x with $1 \leq x \leq n$ is as small as possible. Then either $\lambda_1(\pi|_x) + \lambda_2(\pi|_x) > \lambda_1(\sigma_x) + \lambda_2(\sigma_x)$, or $\lambda_1(\pi|_x) + \lambda_2(\pi|_x) = \lambda_1(\sigma_x) + \lambda_2(\sigma_x)$ and $\sum C_j^{(A)}(\sigma_x) > \sum C_j^{(A)}(\pi|_x)$. In both possibilities, we have

$$\lambda_1(\pi|_x) + \lambda_2(\pi|_x) \geq \lambda_1(\sigma_x) + \lambda_2(\sigma_x) \quad (11)$$

and $\sigma_x \neq \pi|_x$. By the minimality of x , we have $x \geq 2$ and $\sigma_x(x) \neq \pi|_x(x)$. Let $u = \pi|_x(x)$ and $v = \sigma_x(x)$. Then u is the last task in $\pi|_x$ and $\pi|_x[v] < \pi|_x[u]$, and v is the last task in σ_x and $\sigma_x[u] < \sigma_x[v]$. From (1), we have $C_u(\pi|_x) = C_v(\sigma_x)$. From the feasibility of $\pi|_x$ and σ_x , we have $x \leq \min\{k_u, k_v\}$ and, if x' is a B -task with $x' \in \{u, v\}$, then $f_{x'}^{(B)}(C_{x'}(\pi|_x)) \leq Q$. We continue the discussion by distinguishing the following cases.

• u is a task in $\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$. In this case, we define σ' to be the schedule obtained from σ_x by shifting task u to the position just after task v . It can be verified that σ' is a feasible schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_x$. This further implies that $\sigma'|_{x-1}$ is a feasible schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_{x-1}$.

If there exist no A -tasks j such that $\sigma_x[u] < \sigma_x[j] \leq \sigma_x[v]$, then we have

$$\begin{cases} \sum C_j^{(A)}(\sigma'|_{x-1}) &= \sum C_j^{(A)}(\sigma_x), \\ \sum C_j^{(A)}(\pi|_{x-1}) &= \sum C_j^{(A)}(\pi|_x), \\ \lambda_1(\sigma'|_{x-1}) + \lambda_2(\sigma'|_{x-1}) &= \lambda_1(\sigma_x) + \lambda_2(\sigma_x), \\ \lambda_1(\pi|_{x-1}) + \lambda_2(\pi|_{x-1}) &= \lambda_1(\pi|_x) + \lambda_2(\pi|_x). \end{cases}$$

Then one of statements (ii) and (iii) is violated for $\pi|_{x-1}$ and $\sigma'|_{x-1}$. This contradicts the minimality of x .

Now we suppose that there exists at least one A -task j such that $\sigma_x[u] < \sigma_x[j] \leq \sigma_x[v]$. Then we have

$$\begin{cases} \sum C_j^{(A)}(\sigma'|_{x-1}) &< \sum C_j^{(A)}(\sigma_x), \\ \sum C_j^{(A)}(\pi|_{x-1}) &= \sum C_j^{(A)}(\pi|_x), \\ \lambda_1(\pi|_{x-1}) + \lambda_2(\pi|_{x-1}) &= \lambda_1(\pi|_x) + \lambda_2(\pi|_x), \\ \lambda_1(\sigma'|_{x-1}) + \lambda_2(\sigma'|_{x-1}) &< \lambda_1(\sigma_x) + \lambda_2(\sigma_x). \end{cases} \quad (12)$$

Since $\pi|_{x-1}$ is an optimal schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_{x-1}$, from the minimality of x , statement (ii) holds for $\pi|_{x-1}$ and $\sigma'|_{x-1}$, and so, $\lambda_1(\pi|_{x-1}) + \lambda_2(\pi|_{x-1}) \leq \lambda_1(\sigma'|_{x-1}) + \lambda_2(\sigma'|_{x-1})$. From (12), we conclude that $\lambda_1(\pi|_x) + \lambda_2(\pi|_x) < \lambda_1(\sigma_x) + \lambda_2(\sigma_x)$, contradicting (11).

- u is a task in $\mathcal{T}^{(A)}$ and v is a task in $\mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)}$. In this case, we notice that $\pi|_x$ is an optimal schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_x$. By Lemma 3.1, this case also cannot occur.

- Both u and v are tasks in $\mathcal{T}^{(A)}$. In this case, from Algorithm 2, we have that $v < u$ and $\alpha_v \leq \alpha_u$. Let σ'' be the schedule obtained from σ_x by exchanging the positions of tasks u and v . It can be verified that σ'' is still a feasible schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_x$. This further implies that $\sigma''|_{x-1}$ is a feasible schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_{x-1}$. Since $\alpha_v \leq \alpha_u$, we have $p_v(S_v(\sigma'')) = \alpha_v(a + bS_u(\sigma_x)) \leq \alpha_u(a + bS_u(\sigma_x)) = p_u(S_u(\sigma_x))$. Since $u, v \in \mathcal{T}^{(A)}$, $u > v$ and $p_u(S_u(\sigma_x)) \geq p_v(S_v(\sigma''))$, we have

$$\lambda_1(\sigma'') < \lambda_1(\sigma_x) \text{ and } \lambda_2(\sigma'') = \lambda_2(\sigma_x). \quad (13)$$

Since $\pi|_{x-1}$ is an optimal schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ on instance $\mathcal{I}_{x-1}$, the minimality of x implies that statement (ii) holds for $\pi|_{x-1}$ and $\sigma''|_{x-1}$. Thus, we have

$$\lambda_1(\pi|_{x-1}) + \lambda_2(\pi|_{x-1}) \leq \lambda_1(\sigma''|_{x-1}) + \lambda_2(\sigma''|_{x-1}). \quad (14)$$

Moreover,

$$\begin{cases} \lambda_1(\pi|_x) = \lambda_1(\pi|_{x-1}) + |\{j \in \mathcal{T}^{(A)} : j > u, \pi|_x[j] < \pi|_x[u]\}|, \\ \lambda_2(\pi|_x) = \lambda_2(\pi|_{x-1}) + |\{j \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)} : \pi|_x[j] < \pi|_x[u]\}|, \\ \lambda_1(\sigma'') = \lambda_1(\sigma''|_{x-1}) + |\{j \in \mathcal{T}^{(A)} : j > u, \pi|_x[j] < \pi|_x[u]\}|, \\ \lambda_2(\sigma'') = \lambda_2(\sigma''|_{x-1}) + |\{j \in \mathcal{T}^{(B)} \setminus \mathcal{T}^{(A)} : \pi|_x[j] < \pi|_x[u]\}|. \end{cases} \quad (15)$$

Therefore, from Eqs. (13)–(15), we have $\lambda_1(\pi|_x) + \lambda_2(\pi|_x) < \lambda_1(\sigma_x) + \lambda_2(\sigma_x)$, contradicting Eq. (11). The lemma follows. $\square$

Lemma 4.2. The number of nondominated points for problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ (if it is feasible) is at most $\frac{n_A(n_A-1)}{2} + n_A(n - n_A) + 1 = O(n^2)$.

Proof. Suppose that problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ is feasible and has totally N nondominated point $(C_i^{(A)}, F_i^{(B)})$ ($1 \leq i \leq N$) so that $C_i^{(A)} < C_{i+1}^{(A)}$ and $F_i^{(B)} > F_{i+1}^{(B)}$ for $i = 1, \dots, N-1$. Let $F_0^{(A)} = +\infty$. For each i with $1 \leq i \leq N$, let π_i be the schedule obtained from Algorithm 2 with Q being replaced by $F_{i-1}^{(A)} - 1$. By Lemma 4.1(i), π_i is an optimal schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq F_{i-1}^{(A)} - 1$. Since $(C_i^{(A)}, F_i^{(B)})$ is a nondominated point, π_i is a nondominated schedule corresponding to $(C_i^{(A)}, F_i^{(B)})$, $1 \leq i \leq N$. For each i with $1 \leq i \leq N-1$, $C_i^{(A)} < C_{i+1}^{(A)}$ and $F_i^{(B)} > F_{i+1}^{(B)}$ imply that π_{i+1} is a feasible schedule for problem $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq F_{i-1}^{(A)} - 1$. Then $\sum C_j^{(A)}(\pi_i) = C_i^{(A)} < C_{i+1}^{(A)} = \sum C_j^{(A)}(\pi_{i+1})$. From Lemma 4.1(iii), we have $\lambda_1(\pi_i) + \lambda_2(\pi_i) < \lambda_1(\pi_{i+1}) + \lambda_2(\pi_{i+1})$. This implies that $0 \leq \lambda_1(\pi_1) + \lambda_2(\pi_1) < \lambda_1(\pi_2) + \lambda_2(\pi_2) < \dots < \lambda_1(\pi_N) + \lambda_2(\pi_N) \leq \frac{n_A(n_A-1)}{2} + n_A(n - n_A)$, where the last inequality follows from (10). It follows that $N \leq \frac{n_A(n_A-1)}{2} + n_A(n - n_A) + 1 = O(n^2)$. The lemma follows. $\square$

From Lemmas 2.1, 3.2, 3.3, 4.1, and 4.2, we finally have the following result:

Theorem 4.1. Problem $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$ can be solved in $O(n^4)$ time.

As a consequence of Lemma 3.5 and Theorem 4.1, we have the following result:

Theorem 4.2. Problem $1|ND, p_j(t)|^\#(\sum C_j^{(A)}, \tilde{L}_{\max}^{(B)})$ can be solved in $O(n^3 \log n)$ time, where $\tilde{L}_{\max}^{(B)} \in \{C_{\max}^{(B)}, WC_{\max}^{(B)}, L_{\max}^{(B)}, T_{\max}^{(B)}\}$.

5. Conclusions

In this paper, we investigated the ND-agent scheduling of linear-deteriorating tasks on a single machine with positional due indices. The scheduling criteria studied in this paper are the total completion time of the A -tasks $\sum C_j^{(A)}$ and the maximum cost of the B -tasks $f_{\max}^{(B)}$. We provided polynomial-time algorithms for two problems $1|ND, k_j, p_j(t)| \sum C_j^{(A)} : f_{\max}^{(B)} \leq Q$ and $1|ND, k_j, p_j(t)|^\#(\sum C_j^{(A)}, f_{\max}^{(B)})$, respectively. In particular, if $f_{\max}^{(B)}$ is restricted to some lateness-like criteria, for the version without the positional restriction, the time complexity for solving the above two problems can be reduced to $O(n \log n)$ and $O(n^3 \log n)$, respectively.

For further research, we suggest the following topic worthy of study: Linear-deteriorating is just a special version among the rich forms of time-dependent deteriorating models. One may generally study the ND-agent scheduling of time-dependent tasks on a single machine or parallel machines with positional due indices. Since there are many valuable problems to be studied, this research topic will be fruitful.

Acknowledgment

The authors would like to thank the associate editor and four anonymous referees for their constructive comments and helpful suggestions. This research was supported by NSFC under grant numbers 11671368, 11571321 and 11771406.

References

- [1] A. Agnetis, J.C. Billaut, S. Gawiejnowicz, D. Pacciarelli, A. Soukhal, *Multiagent Scheduling: Models and Algorithms*, Springer, Berlin, 2014.
- [2] A. Agnetis, P.B. Mirchandani, D. Pacciarelli, A. Pacifici, Scheduling problems with two competing agents, *Oper. Res.* 52 (2004) 229–242.
- [3] A. Agnetis, D. Pacciarelli, A. Pacifici, Multi-agent single machine scheduling, *Ann. Oper. Res.* 150 (2007) 3–15.
- [4] B. Alidaee, N.K. Womer, Scheduling with time dependent processing times: review and extensions, *J. Oper. Res. Soc.* 50 (1999) 711–720.
- [5] A. Bachman, A. Janiak, Scheduling jobs with special type of start time dependent processing times, Wroclaw University of Technology, 1997 Report pre.
- [6] K.R. Baker, J.C. Smith, A multiple-criterion model for machine scheduling, *J. Sched.* 6 (2003) 7–16.
- [7] S. Browne, U. Yechiali, Scheduling deteriorating jobs on a single processor, *Oper. Res.* 38 (1990) 495–498.
- [8] T.C.E. Cheng, Q. Ding, B.M. Lin, A concise survey of scheduling with time-dependent processing times, *Eur. J. Oper. Res.* 152 (2004) 1–13.
- [9] T.C.E. Cheng, C.T. Ng, J.J. Yuan, Multi-agent scheduling on a single machine to minimize total weighted number of tardy jobs, *Theor. Comput. Sci.* 362 (2006) 273–281.
- [10] T.C.E. Cheng, C.T. Ng, J.J. Yuan, Multi-agent scheduling on a single machine with max-form criteria, *Eur. J. Oper. Res.* 188 (2008) 603–609.
- [11] Y. Gao, J.J. Yuan, Pareto minimizing total completion time and maximum cost with positional due indices, *J. Oper. Res. Soc. China* 3 (2015) 381–387.
- [12] Y. Gao, J.J. Yuan, Bi-criteria Pareto-scheduling on a single machine with due-indices and precedence constraints, *Discr. Optim.* 25 (2017) 105–119.
- [13] Z.C. Geng, J.J. Yuan, Pareto optimization scheduling of family jobs on a p-batch machine to minimize makespan and maximum lateness, *Theor. Comput. Sci.* 570 (2015) 22–29.
- [14] C. He, J.Y.T. Leung, Two-agent scheduling of time-dependent jobs, *J. Comb. Optim.* 34 (2017) 362–377.
- [15] H. Hoogeveen, Multicriteria scheduling, *Eur. J. Oper. Res.* 167 (2005) 592–623.
- [16] N. Huynh-Tuong, A. Soukhal, Single machine scheduling with interfering jobs, in: *Proceedings of the International Conference on Project Management and Scheduling*, PMS Tours, France, 2010, pp. 243–247.
- [17] N. Huynh-Tuong, A. Soukhal, J.C. Billaut, Single-machine multi-agent scheduling problems with a global objective function, *J. Sched.* 15 (2012) 311–321.
- [18] A. Kononov, S. Gawiejnowicz, NP-hard cases in scheduling deteriorating jobs on dedicated machines, *J. Oper. Res. Soc.* 52 (2001) 708–717.
- [19] J.Y.T. Leung, M. Pinedo, G.H. Wan, Competitive two-agent scheduling and its applications, *Oper. Res.* 58 (2010) 458–469.
- [20] P. Liu, L.X. Tang, Two-agent scheduling with linear deteriorating jobs on a single machine, *Comput. Comb.* (2008) 642–650.
- [21] P. Liu, N. Yi, X. Zhou, Two-agent single-machine scheduling problems under increasing linear deterioration, *Appl. Math. Model.* 35 (2011) 2290–2296.
- [22] O.I. Melnikov, Y.M. Shafirsky, Parametric problem of scheduling theory, *Cybernetics* 15 (1979) 352–357.
- [23] C.T. Ng, T.C.E. Cheng, J.J. Yuan, A note on the complexity of the problem of two-agent scheduling on a single machine, *J. Comb. Opt.* 12 (2006) 87–394.
- [24] C.T. Ng, J.B. Wang, T.C.E. Cheng, L.L. Liu, A branch-and-bound algorithm for solving a two-machine flow shop problem with deteriorating jobs, *Comput. Oper. Res.* 37 (2010) 83–90.
- [25] V. Strusevich, K. Rustogi, *Scheduling with Times-Changing Effects and Rate-Modifying Activities*, Springer, 2017.
- [26] P. Perez-Gonzalez, J.M. Framinan, A common framework and taxonomy for multicriteria scheduling problems with interfering and competing jobs: Multi-agent scheduling problems, *Eur. J. Oper. Res.* 235 (2013) 1–16.
- [27] V. T'Kindt, J.C. Billaut, *Multicriteria Scheduling: Theory, Models and Algorithms*, Springer, Berlin, 2006.
- [28] J.B. Wang, C.T. Ng, T.C.E. Cheng, Single-machine scheduling with deteriorating jobs under a series-parallel graph constraint, *Comput. Oper. Res.* 35 (2008a) 2684–2693.
- [29] J.B. Wang, L. Lin, F. Shan, Single-machine group scheduling problems with deteriorating jobs, *Int. J. Adv. Manuf. Technol.* 39 (2008b) 808–812.
- [30] J.B. Wang, J.J. Wang, P. Ji, Scheduling jobs with chain precedence constraints and deteriorating jobs, *J. Oper. Res. Soc.* 62 (2011) 1765–1770.
- [31] J.B. Wang, M.Z. Wang, Minimizing makespan in three-machine flow shops with deteriorating jobs, *Comput. Oper. Res.* 40 (2013) 547–557.
- [32] J.B. Wang, Z.Q. Xia, Flow shop scheduling problems with deteriorating jobs under dominating machines, *J. Oper. Res. Soc.* 57 (2006) 220–226.
- [33] Z. Wang, C.M. Wei, Y.B. Wu, Single machine two-agent scheduling with deteriorating jobs, *Asia-Pac. J. Oper. Res.* 33 (2016) 1650034.
- [34] Y. Yin, T.C.E. Cheng, L. Wan, C.C. Wu, J. Liu, Two-agent single-machine scheduling with deteriorating jobs, *Comput. Ind. Eng.* 81 (2015) 177–185.
- [35] J.J. Yuan, Complexities of some problems on multi-agent scheduling on a single machine, *J. Oper. Res. Soc. China* 4 (2016) 379–384.
- [36] J.J. Yuan, W.P. Shang, Q. Feng, A note on the scheduling with two families of jobs, *J. Sched.* 8 (2005) 537–542.
- [37] J.J. Yuan, Unary NP-hardness of minimizing the number of tardy jobs with deadlines, *J. Sched.* 20 (2017a) 211–218.
- [38] J.J. Yuan, Multi-agent scheduling on a single machine with a fixed number of competing agents to minimize the weighted sum of number of tardy jobs and makespans, *J. Comb. Opt.* 34 (2017b) 433–440.
- [39] J.J. Yuan, Complexities of four problems on two-agent scheduling, *Optim. Lett.* 12 (2018) 763–780.
- [40] J.J. Yuan, C.T. Ng, T.C.E. Cheng, Scheduling with release dates and preemption to minimize multiple max-form objective functions, *Eur. J. Oper. Res.* (2019), doi:10.1016/j.ejor.2019.07.072.
- [41] C.L. Zhao, Q.L. Zhang, H.Y. Tang, Scheduling problems under linear deterioration, *Acta Autom. Sin.* 29 (2003) 531–535.
- [42] Q.L. Zhao, L.F. Lu, J.J. Yuan, Rescheduling with new orders and general maximum allowable time disruptions, *4OR Q. J. Oper. Res.* 14 (2016) 261–280.
- [43] Q.L. Zhao, J.J. Yuan, Rescheduling to minimize the maximum lateness under the sequence disruptions of original jobs, *Asia-Pac. J. Oper. Res.* 34 (2017) 1750024.